

Numerical Analysis Assignment 1

Computational Component

Description

This assignment consists of two independent programming problems. In Problem 1 you will build a general-purpose symbolic Taylor series generator using `sympy`. In Problem 2 you will implement a robust, production-quality Bisection Method root-finder that supports multiple error/stopping criteria and defensive error handling. Both problems require more than a "textbook" implementation: you are expected to anticipate edge cases, validate inputs, and structure your code so that it could plausibly be reused as a small library.

General Requirements

- Use **Python 3.8+**. Allowed libraries: `sympy`, `numpy`, `matplotlib`. No other numerical/root-finding libraries (e.g. `scipy.optimize`) may be used — the point of the assignment is to implement the underlying algorithms yourself.
- Every function must include a complete **docstring** (parameters, return values, exceptions raised, and a short example).
- Your code must be organized into two files: `taylor_series.py` and `bisection_method.py`, plus a short `report.pdf/report.md` discussing your results (see §3).
- Include a `if __name__ == "__main__":` block in each file that demonstrates your function(s) on the required test cases below.
- (Optional) Do not let your program crash on bad input. Anticipated failure modes must raise **informative, specific exceptions** (not bare `Exception`), or return a clearly documented sentinel/result object — your choice, but be consistent and explain your choice in the report.

1 Problem 1: Symbolic Taylor Series Generator (50 points)

1.1 Background

Given a function f , an expansion point x_0 , and an order n , the n -th Taylor polynomial is

$$P_n(x) = \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k,$$

with Lagrange remainder $R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)^{n+1}$ for some ξ between x_0 and x . Rather than approximating this numerically, you will construct the polynomial *symbolically*, term by term, using differentiation rules encoded in `sympy` — you may **not** call `sympy.series()` or `sympy.Function.series` directly, since that would trivialize the assignment. You must build $P_n(x)$ yourself using repeated symbolic differentiation (`sympy.diff`), substitution (`.subs`), and `sympy.factorial`.

1.2 Task

Implement a function with (at minimum) the following signature:

```
1 def taylor_series(expr, point, n, var=None):
2     """
3     Construct the n-th order Taylor polynomial of 'expr' about 'point',
4     symbolically, using term-by-term differentiation.
5
6     Parameters
7     -----
8     expr : sympy.Expr or str
9         The function to expand, e.g. sympy.exp(x) or "exp(x)*sin(x)".
10    point : int, float, or sympy number
11        The expansion point x0.
12    n : int
13        The order of the Taylor polynomial (must satisfy n >= 0).
14    var : sympy.Symbol, optional
15        The variable of expansion. If None, it must be inferred
16        automatically from 'expr' (raise an error if 'expr' has zero
17        or more than one free symbol and var was not supplied).
18
19    Returns
20    -----
21    dict
22    {
23        "polynomial": sympy.Expr,          # P_n(x), fully simplified
24        "terms": list[sympy.Expr],         # individual terms, term[k] is
25                                           # the k-th order contribution
26        "remainder": sympy.Expr,          # symbolic R_n(x) with a fresh
27                                           # symbol xi for the unknown
28                                           # point
29        "latex": str                      # LaTeX string of the
30                                           polynomial
31    }
32
33    Raises
```

```

32 -----
33 ValueError
34     If  $n < 0$ , if 'var' cannot be inferred, or if 'expr' is not
35      $n+1$  times differentiable at a symbolic level (e.g. sympy
36     cannot compute the derivative).
37     """

```

Required Functionality

- (1) **Term-by-term construction.** Compute each derivative $f^{(k)}(x)$ for $k = 0, \dots, n$ using `sympy.diff`, evaluate it at x_0 with `.subs`, divide by $k!$, and multiply by $(x - x_0)^k$. Store each term separately in the "terms" list before summing.
- (2) **Symbolic remainder.** Construct $R_n(x)$ using a *new, distinct* symbolic variable (e.g. `xi = sympy.Symbol('xi')`) representing the unknown point guaranteed by Taylor's Theorem. This should be returned as an unevaluated symbolic expression, not a number.
- (3) **Automatic variable inference.** If `var` is not provided, use `expr.free_symbols` to determine the variable of expansion automatically, and raise a `ValueError` with a clear message if this is ambiguous (0 or > 1 free symbols).
- (4) **Numerical evaluation helper.** Implement a second function `evaluate_and_compare(expr, point, n, x_eval, var=None)` that:
 - builds P_n via `taylor_series`,
 - evaluates $P_n(x_eval)$ and the true $f(x_eval)$ numerically (`sympy.N` or `float`),
 - computes the actual absolute and relative error, and
 - returns a dictionary with all four numbers (`approx`, `true_value`, `abs_error`, `rel_error`).
- (5) **Visualization.** Implement `plot_taylor_approximations(expr, point, orders, x_range, var=None)` that plots $f(x)$ together with $P_n(x)$ for each n in the list `orders` (e.g. `[1, 2, 4, 8]`) over `x_range`, with a legend, using `matplotlib`. Save the figure to `taylor_plot.png`.

Edge Cases (Optional)

- $n = 0$ (should return the constant term $f(x_0)$ only, with an empty higher-order term list beyond the zeroth term).
- A negative or non-integer n (raise `ValueError`).
- An expression with *no* free symbols, e.g. `expr = sympy.Integer(5)` (raise a clear `ValueError` since there is nothing to expand).
- An expression with more than one free symbol and no `var` argument supplied (e.g. `x*y`) — must raise `ValueError` rather than silently guessing.

- A function that is not differentiable at the expansion point in the ordinary sense, e.g. `sympy.Abs(x)` expanded about $x_0 = 0$, or `sympy.sqrt(x)` about $x_0 = 0$. Your function should either raise an informative error or clearly document sympy's behavior (e.g. sympy may return `zoo/nan`) — detect this case and raise `ValueError` instead of silently returning garbage.
- Expansion about a *non-zero* point, e.g. $\ln(x)$ about $x_0 = 1$, and about a *symbolic* point, e.g. leaving x_0 as a symbol a rather than a number (this should still work and return a polynomial in terms of a).

1.3 Required Test Cases

Your `_main_` block must demonstrate, at minimum:

- $f(x) = e^x$ about $x_0 = 0$, orders $n = 1, 3, 5$; compare $P_n(0.5)$ to the true value of $e^{0.5}$ and report absolute/relative error for each order.
- $f(x) = \sin(x)$ about $x_0 = 0$, order $n = 7$; verify symbolically that all even-order terms vanish (print the `"terms"` list and confirm this).
- $f(x) = \ln(x)$ about $x_0 = 1$, order $n = 4$; evaluate $P_4(1.2)$ and compare to $\ln(1.2)$.
- $f(x) = \frac{1}{1-x}$ about $x_0 = 0$, order $n = 5$; evaluate at $x = 0.9$ and at $x = 1.5$, and comment in your report on why the approximation is excellent in one case and poor (or divergent) in the other, tying this back to the radius of convergence.
- One deliberately invalid call for each edge case listed above, demonstrating that your function raises the expected exception (use `try/except` and print a confirmation message rather than letting the program crash).
- The visualization function called on $f(x) = \cos(x)$ about $x_0 = 0$ with `orders=[1,2,4,6]` over $x \in [-2\pi, 2\pi]$.

2 Problem 2: Bisection Method Solver

2.1 Background

The Bisection Method locates a root of a continuous function f on $[a, b]$ with $f(a)f(b) < 0$ by repeatedly halving the interval. A *correct* implementation is easy; a *robust* one must validate its inputs, support multiple stopping criteria, and give the caller diagnostic information about *why* and *when* it stopped.

2.2 Task

Implement a function with (at minimum) the following signature:

```
1 def bisection(f, a, b, tol=1e-8, max_iter=100,
2             error_type="absolute", func=None, verbose=False):
3     """
4     Find a root of f on [a, b] using the Bisection Method.
5
6     Parameters
7     -----
8     f : callable
9         A function of one variable, f(x) -> float.
10    a, b : float
11        Endpoints of the initial bracketing interval, a < b.
12    tol : float
13        The tolerance for the chosen stopping criterion. Must be > 0.
14    max_iter : int
15        Maximum number of iterations allowed before giving up.
16    error_type : str
17        One of:
18        "absolute" -> stop when |p_n - p_{n-1}| < tol
19        "relative" -> stop when |p_n - p_{n-1}| / |p_n| < tol
20        "residual" -> stop when |f(p_n)| < tol
21        "interval" -> stop when (b_n - a_n) < tol
22        "all" -> stop only when ALL of the above (with the
23                same tol) are satisfied simultaneously
24    verbose : bool
25        If True, print a formatted iteration table as it runs.
26
27    Returns
28    -----
29    dict
30        {
31            "root": float,                # final approximation p_n
32            "iterations": int,            # number of iterations
33                                     performed
34            "converged": bool,            # True if stopping tol was met
35                                     # before max_iter
36            "reason": str,                # human-readable reason for
37            "history": list[dict],        # one dict per iteration with
38                                     # keys: n, a, b, p, f_p,
39                                     # abs_error, rel_error,
39                                     interval_width
```

```

39         "a_priori_iterations": int          # ceil(log2((b-a)/tol)), for
40                                             # comparison against '
41                                             iterations'
42     }
43
44     Raises
45     -----
46     ValueError
47         If a >= b, tol <= 0, max_iter <= 0, error_type is invalid,
48         or f(a) and f(b) do not have opposite signs (f(a)*f(b) >= 0).
49     TypeError
50         If f is not callable.
51     """

```

Required Functionality

- (1) **Full input validation** before any iteration begins: type-check `f` (callable), check `a < b`, check `tol > 0`, check `max_iter` is a positive integer, check `error_type` is one of the five allowed strings, and check $f(a) \cdot f(b) < 0$ — with a distinct, specific error message for each failure mode (do not lump them into one generic message).
- (2) **Special-case short-circuits:** if $f(a) = 0$ exactly, return a immediately as the root (similarly for $f(b) = 0$), without performing any bisection iterations. Document this in the returned `"reason"` field.
- (3) **Four independent stopping criteria plus a combined "all" mode**, exactly as specified in the docstring. Track *all four* quantities (absolute error, relative error, residual, interval width) at every iteration regardless of which `error_type` is active, and store them in `history` so the user can analyze the run afterward no matter which criterion triggered the stop.
- (4) **Max-iteration handling:** if `max_iter` is reached without satisfying the stopping criterion, do *not* raise an exception — return normally with `"converged": False` and an explanatory `"reason"`, so the caller can decide how to proceed.
- (5) **A priori iteration count:** compute and return $\lceil \log_2((b-a)/\text{tol}) \rceil$ for comparison, regardless of `error_type` (note in your report that this formula is derived for the interval-width criterion specifically, and discuss whether it over- or under-predicts the iterations actually needed for the other criteria).
- (6) **Verbose mode:** when `verbose=True`, print a nicely aligned table with columns $n, a_n, b_n, p_n, f(p_n)$, interval width, at every iteration.
- (7) **Convergence plot helper:** implement a second function `plot_convergence(result)` that takes the dictionary returned by `bisection` and produces a semi-log plot (`matplotlib`, `plt.semilogy`) of interval width vs. iteration number, annotated with a reference line of slope corresponding to the theoretical convergence rate $1/2$ per iteration, saved to `bisection_convergence.png`.

Edge Cases and Failure Modes (Optional)

- $a \geq b$ (raise `ValueError`).
- $f(a)$ and $f(b)$ have the *same* sign, e.g. $f(x) = x^2 - 1$ on $[-0.5, 0.5]$ (raise `ValueError` with a message that reports the actual values $f(a)$ and $f(b)$).
- $f(a) = 0$ or $f(b) = 0$ exactly (handled without iterating, per requirement (2) above).
- `tol <= 0` or `max_iter <= 0` (raise `ValueError`).
- An invalid `error_type` string, e.g. "foo" (raise `ValueError` listing the valid options).
- `f` is not callable, e.g. the user accidentally passes a number or a sympy expression instead of a Python function (raise `TypeError` with guidance, e.g. suggesting `sympy.lambdify` if a sympy expression was passed).
- A function with *more than one root* in $[a, b]$, e.g. $f(x) = \sin(4\pi x)$ on $[0, 1]$ — your function should not crash or behave unpredictably; it should simply converge to *some* root, and you must discuss in your report which root it finds and why, given the specific a, b you test.
- **relative** error type when p_n passes extremely close to 0 (potential division by a near-zero number) — decide on and document a safeguard (e.g. adding a small epsilon to the denominator, or falling back to absolute error when $|p_n|$ is below some threshold) and justify your choice in the report.
- `max_iter` set deliberately too low (e.g. `max_iter=3`) for a case that would normally need more iterations — confirm the function returns gracefully with `"converged": False`.

2.3 Required Test Cases

Your `_main_` block must demonstrate, at minimum:

- (i) $f(x) = x^3 - x - 2$ on $[1, 2]$ with `error_type="interval"`, `tol=1e-6`. Print the full result dictionary (excluding the full history list; print only the first and last 3 rows of history) and compare `iterations` to `a_priori_iterations`.
- (ii) The same function and interval, run four separate times with `error_type` $\in \{\text{"absolute", "relative", "residual", "all"}\}$, all with `tol=1e-6`. Present a small comparison table (you may print it or build it with a Python dict/list of tuples) showing the number of iterations required under each criterion.
- (iii) Each of the nine failure modes listed above, triggered deliberately and caught with `try/except`, printing the resulting exception message.
- (iv) $f(x) = \sin(4\pi x)$ on $[0, 1]$ — report which root is found and discuss why in your report (this interval actually contains multiple sign changes; think carefully about whether $f(0) \cdot f(1) < 0$ even holds here, and if not, choose a valid sub-bracket and explain your choice).
- (v) A call to `plot_convergence` for test case (i), and a written comment in your report on whether the empirical slope matches the theoretical rate of $1/2$ per iteration.

3 Deliverables

- (1) `taylor_series.py` — Problem 1 implementation and test cases.
- (2) `bisection_method.py` — Problem 2 implementation and test cases.
- (3) `taylor_plot.png` and `bisection_convergence.png` — generated figures.
- (4) A short **report** (1–3 pages, PDF or Markdown) that:
 - summarizes your design choices (especially for error handling and the relative-error near-zero safeguard),
 - includes the requested numerical comparisons and discussion points from both problems,
 - states any known limitations of your implementation.
- (5) Submit all files in a single archive named `lastname_firstname_assignment.zip`, on LMS by 11:30PM on Tuesday Sep 22, 2026.

4 Academic Integrity

You may use documentation for `sympy`, `numpy`, and `matplotlib`. You may not use any pre-built root-finding or series-expansion routine that performs the core task for you (e.g. `sympy.series`, `scipy.optimize.bisect`). You may discuss approaches with classmates at a conceptual level, but all code must be your own. Any use of AI must be disclosed in sufficient details, including the extent to which it was used and the purpose for which it was used.